



湖南大学
HUNAN UNIVERSITY

第二章 高级数据结构

湖南大学计算机学院

引言



- 对于计算机而言，数组拥有最高的访问效率（无指针跳转、内存连续）。
- 高级数据结构的本质，就是如何巧妙地利用数组来模拟复杂逻辑，从而兼顾性能与功能。
- 课程目标：
 - 堆：如何在数组中快速定位并维护最大值/最小值？
 - 哈希表：如何通过数学映射，在数组中实现常数级检索？

目录

第一节

堆

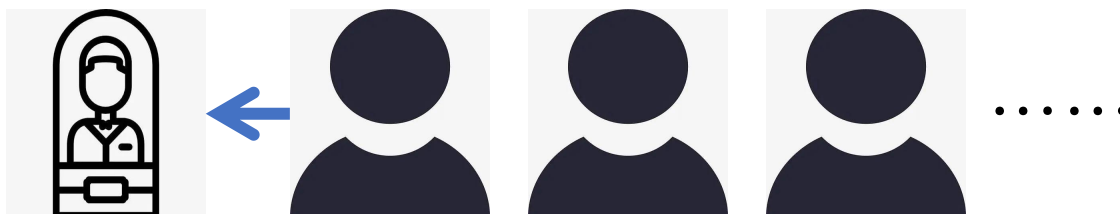
第二节

哈希表

队列



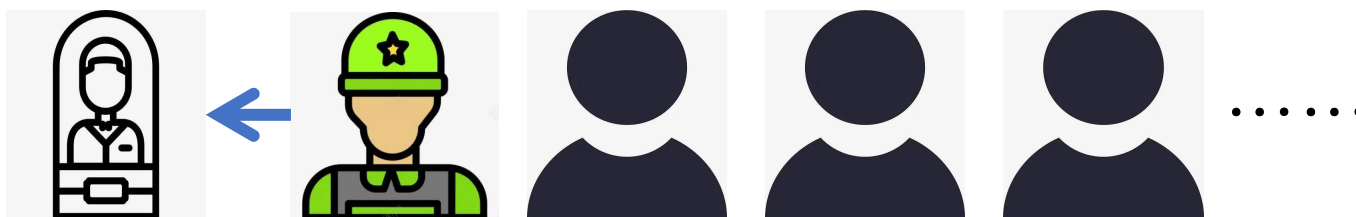
队列： 满足先进先出（FIFO）的数据结构，数据从队列头部取出，新的数据从队列尾部插入，数据之间是平等的。类似于普通人到火车站排队买票。



优先队列



优先队列：从头部取出数据，从尾部插入数据，但是这个过程根据数据的优先级而变化的，总是优先级高的先出来。类似于军人依法优先。



优先队列问题定义



优先队列是一种用来维护由一组元素构成的集合 S 的数据结构，其中每一个元素都有一个关键字(key)，关键字赋予了一个元素的优先级，故名为优先队列。

优先队列需求



优先队列的操作:

- 1) $\text{Insert}(S, x)$: 插入 x 到集合 S 中;
- 2) $\text{Maximum}(S)$: 返回 S 中具有最大 (或者最小) 关键字的元素;
- 3) $\text{Extract_Max}(S)$: 去掉并返回集合 S 中具有最大 (或者最小) 关键字的元素;
- 4) $\text{Increase_Key}(S, x, \text{key})$: 将元素 x 的关键字增加到 key 。

堆的作用



堆(Heap)是一类特殊的数据结构，是基于数组的高效的优先队列实现方式

堆的应用场景：

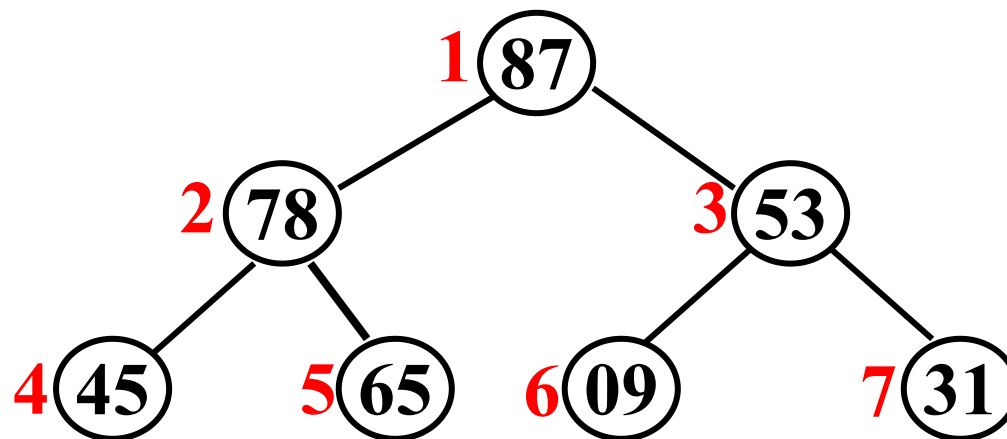
- 优先级队列的使用场景
- 求Top K值问题
- 求中位数、百分位数
- 大数据量日志统计搜索排行榜

课程目标： 堆的定义与操作

堆的定义



满二叉树：一棵深度为 k 且有 2^k-1 个结点的二叉树



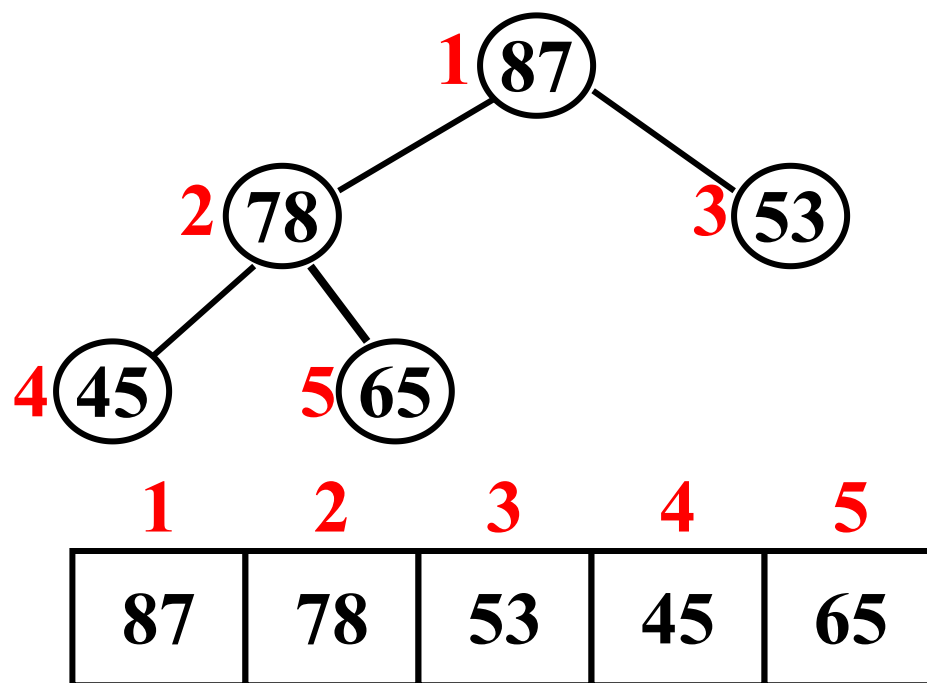
如果对满二叉树的结点进行编号, 约定编号从根结点起, 自上而下, 自左而右, 则可以形成一个长度为 2^k-1 的序列

1	2	3	4	5	6	7
87	78	53	45	65	09	31

堆的定义



完全二叉树：一棵深度为 k 的有 n 个结点的二叉树，对树中的结点按从上至下、从左到右的顺序进行编号，如果编号为 i ($1 \leq i \leq n$) 的结点与满二叉树中编号为 i 的结点在二叉树中的位置相同，则这棵二叉树称为完全二叉树。



堆的定义



堆实质上是满足如下性质的完全二叉树：树中任一非叶结点的关键字均不大于(或不小于)其左右孩子(若存在)结点的关键字

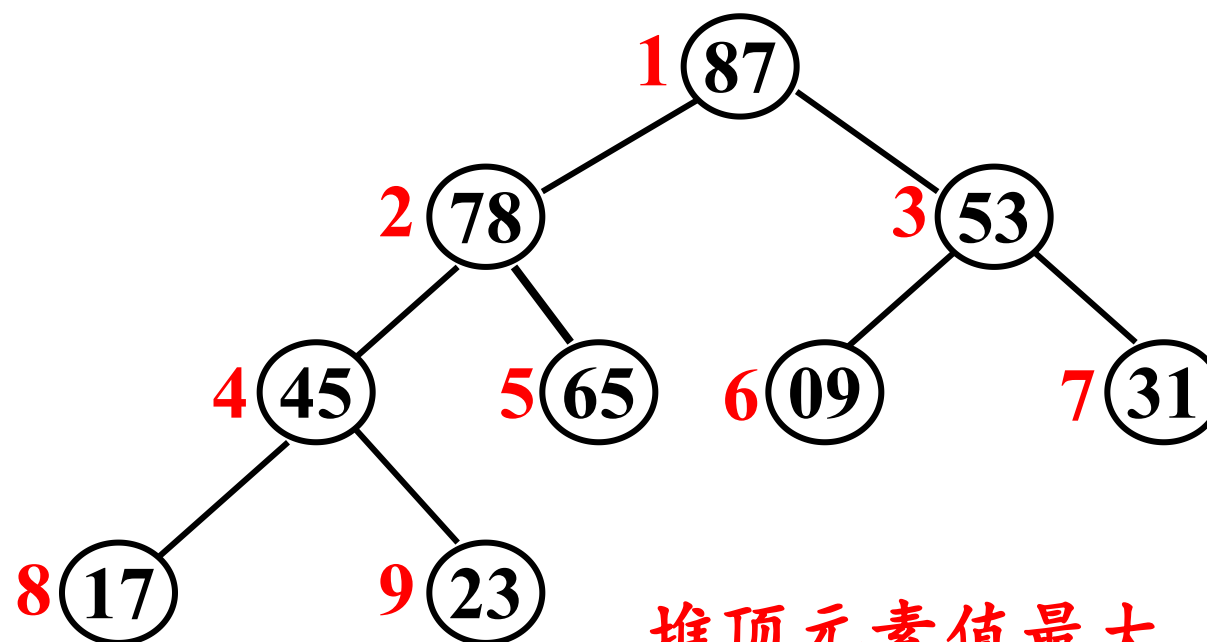
对应序列上， n 个元素的序列 $\{k_1, k_2, \dots, k_n\}$ 当且仅当满足如下关系时，称之为**堆 (heap)**。若满足条件 $k_i \leq k_{i/2}$ 且则称**最大堆**；若满足条件 $k_i \geq k_{i/2}$ 则称**最小堆**

堆的实例



最大堆

1	2	3	4	5	6	7	8	9
87	78	53	45	65	09	31	17	23



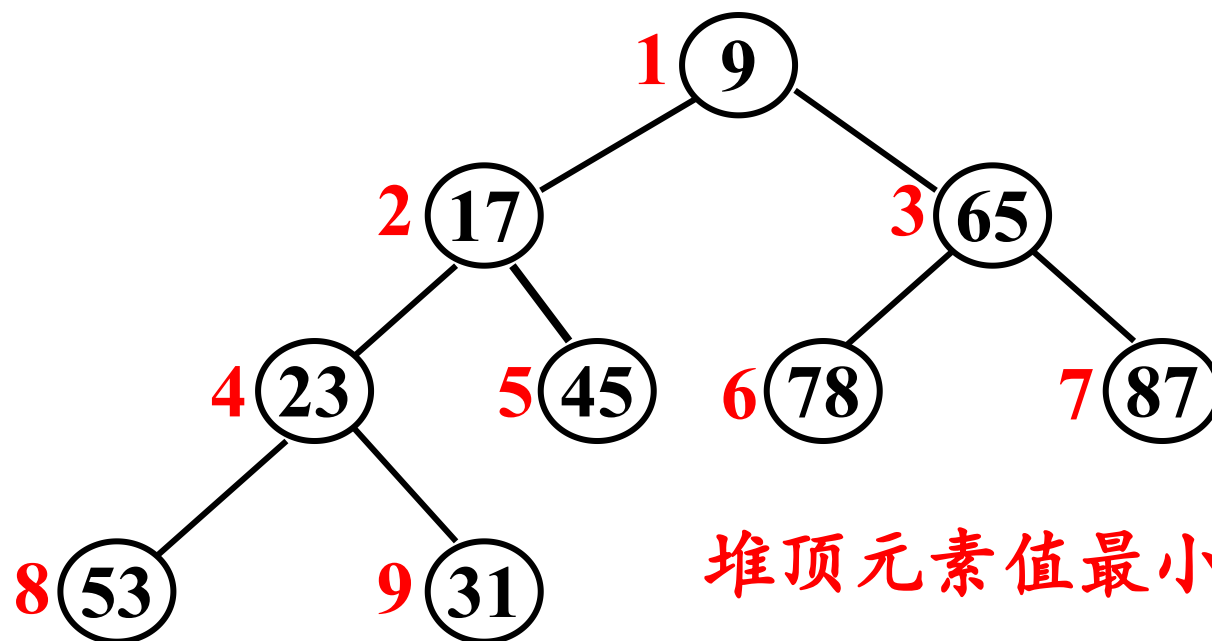
堆顶元素值最大

堆的实例



最小堆

1	2	3	4	5	6	7	8	9
9	17	65	23	45	78	87	53	31



堆顶元素值最小

将堆用顺序存储结构来存储，则堆对应一组序列

优先队列需求



优先队列的操作:

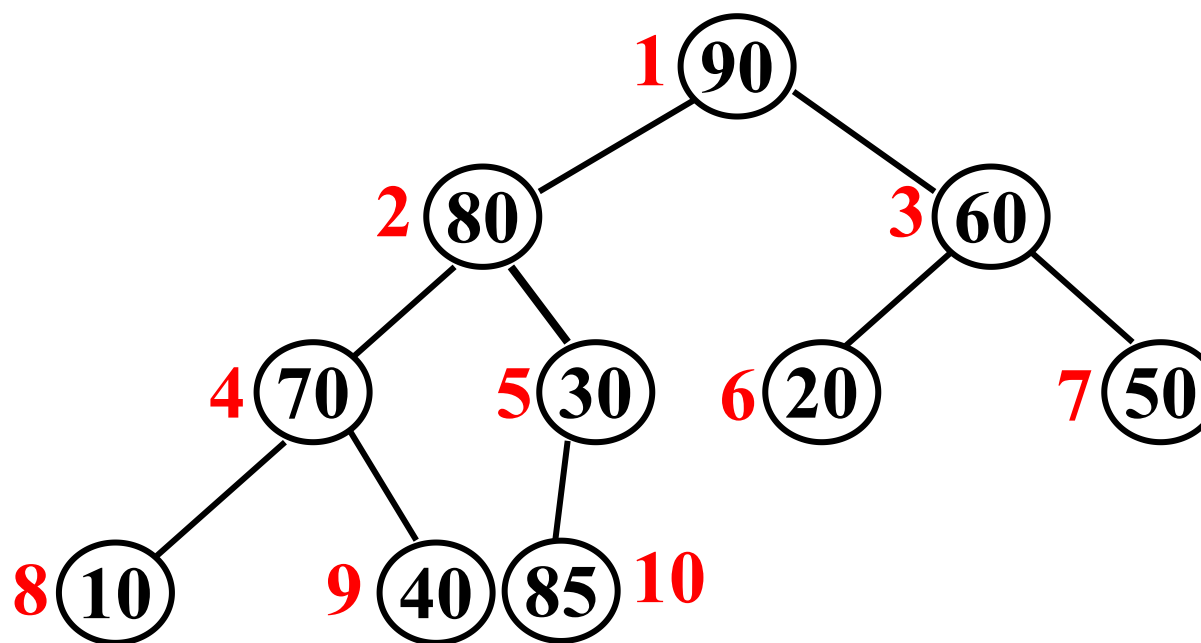
- 1) **Insert(S, x):**插入x到集合S中;
- 2) **Maximum(S):**返回S中具有最大 (或者最小) 关键字的元素;
- 3) **Extract_Max(S):**去掉并返回集合S中具有最大 (或者最小) 关键字的元素;
- 4) **Increase_Key(S, x, key):**将元素x的关键字增加到key。

堆的插入



插入一个新的元素85之后呢？

90	80	60	70	30	20	50	10	40	85			
----	----	----	----	----	----	----	----	----	----	--	--	--



堆的插入



如何在堆中插入一个元素？

解决方法：

在堆的最后增加一个元素，然后新添加的元素不能比它的父元素大，如果比它的父元素大，就涉及到上浮的动作

堆的插入



如何调整成一个堆?

90	80	60	70	30	20	50	10	40	85			
----	----	----	----	----	----	----	----	----	----	--	--	--

90	80	60	70	85	20	50	10	40	30			
----	----	----	----	----	----	----	----	----	----	--	--	--

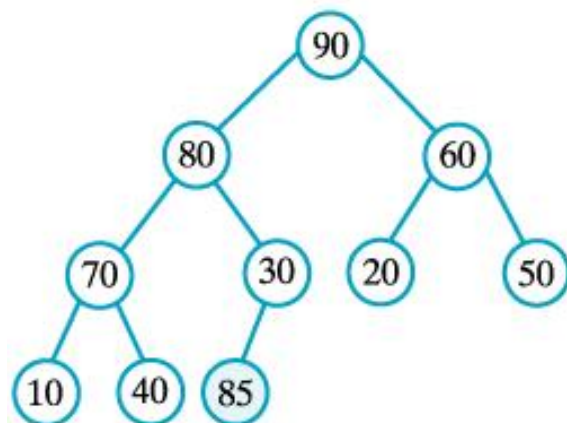
90	85	60	70	80	20	50	10	40	30			
----	----	----	----	----	----	----	----	----	----	--	--	--

堆的插入

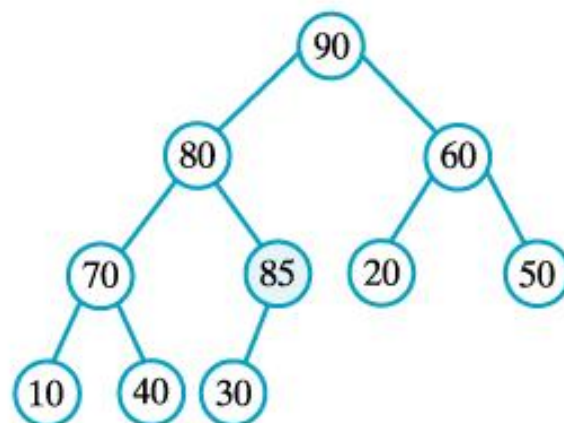


最大堆中加入元素85

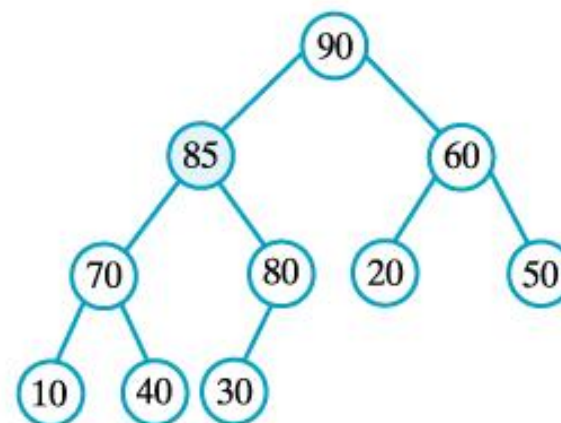
(a)



(b)



(c)





在堆中插入一个元素复杂度

最坏情况就是自底向上上浮到堆的根，于是时间代价为堆的深度 k

因为堆对应完全二叉树，而层数为 k 的完全二叉树最少有 2^{k-1}

于是，对于包含 n 个元素的堆而言

$$2^{k-1} \leq n \Rightarrow k - 1 \leq \log_2 n \Rightarrow k \leq \log_2 n + 1$$

因此，在堆中插入一个元素时间代价为 $O(\log n)$

优先队列需求



优先队列的操作:

- 1) $\text{Insert}(S, x)$: 插入 x 到集合 S 中;
- 2) **$\text{Maximum}(S)$** : 返回 S 中具有最大 (或者最小) 关键字的元素;
- 3) **$\text{Extract_Max}(S)$** : 去掉并返回集合 S 中具有最大 (或者最小) 关键字的元素;
- 4) $\text{Increase_Key}(S, x, \text{key})$: 将元素 x 的关键字增加到 key 。

堆的输出



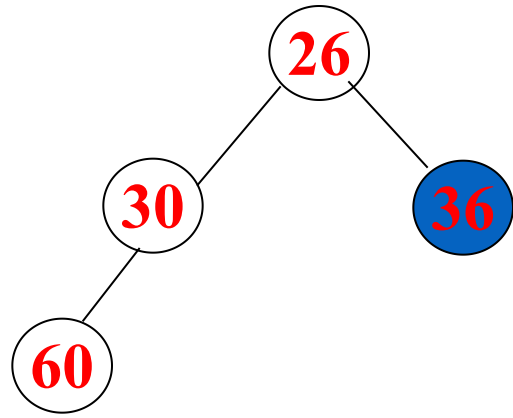
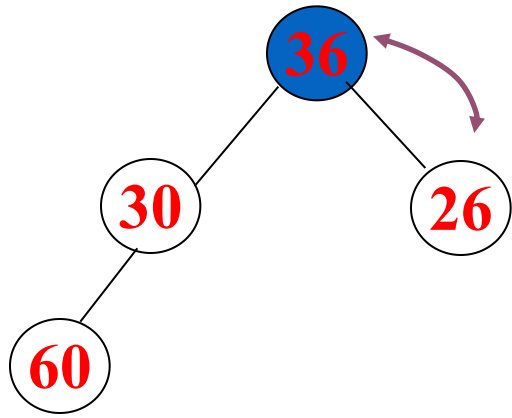
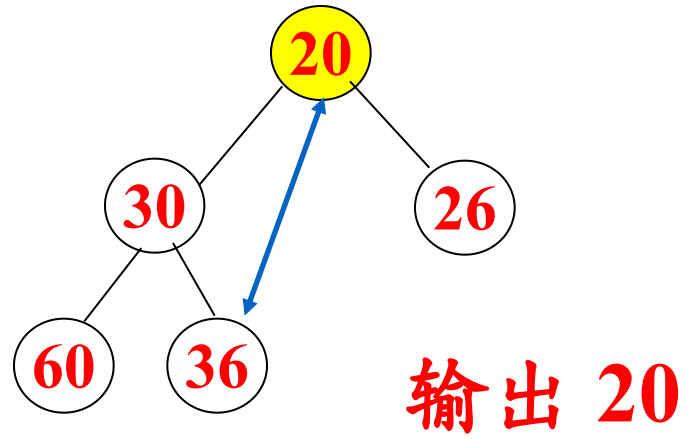
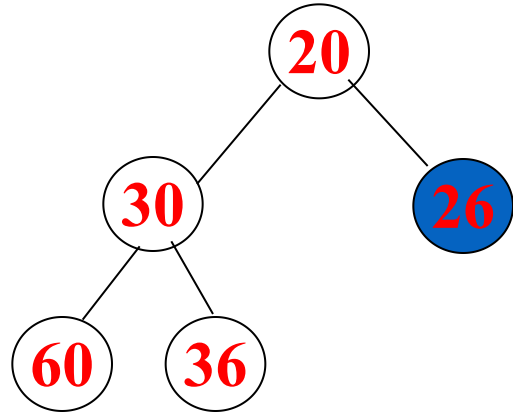
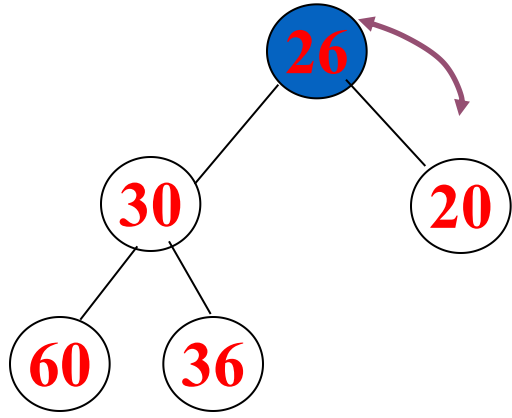
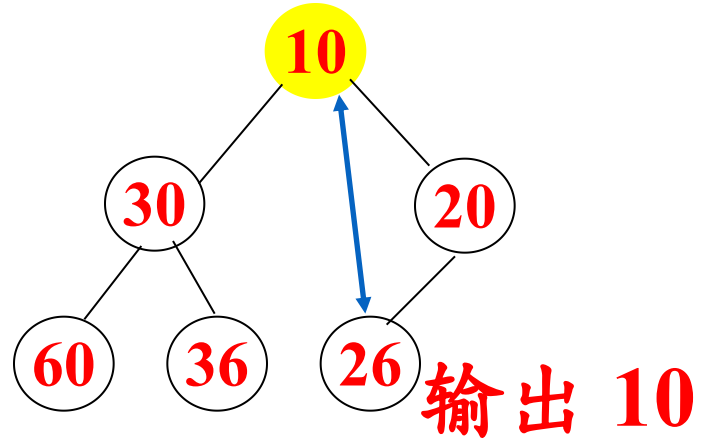
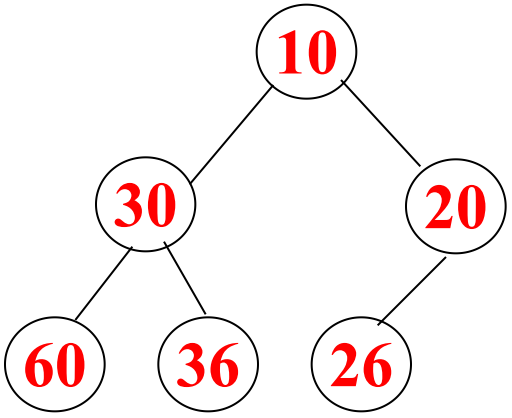
如何从堆中输出最大（或者最小）元素？

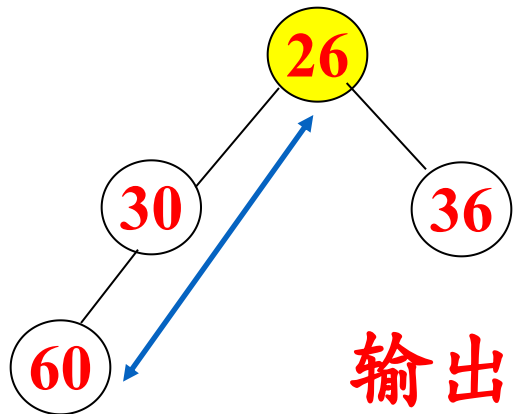
解决方法：

将最大（或者最小）元素输出，然后堆的最后元素放到堆的第一位，然后新的根元素不能比它的父元素小（或者大），如果比它的父元素小（或者大），就涉及到下沉的动作

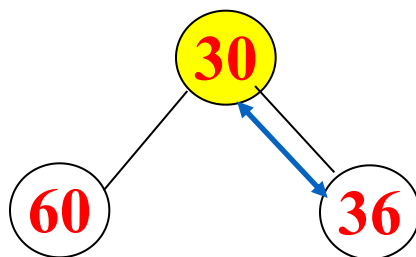
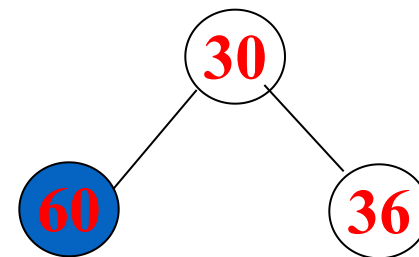
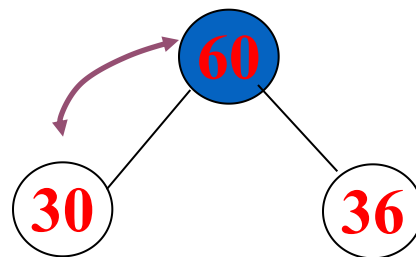
堆的输出

最小堆输出序列

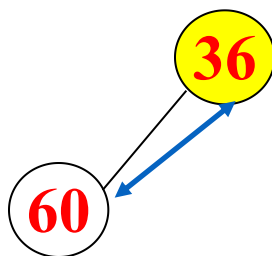
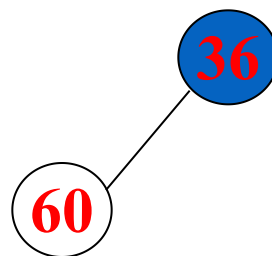




输出 26



输出 30



输出 36

输出 60

递增输出序列为：10 20 26 30 36 60

堆的输出



从堆中输出最大（或者最小）元素复杂度

最坏情况就是新的根自顶向下下沉到堆的叶子，于是时间代价为堆的深度 k

因此，从堆中输出最大（或者最小）元素时间代价为 $O(\log n)$

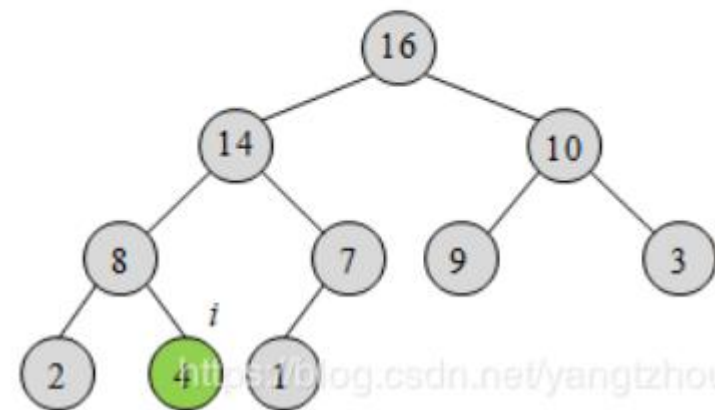
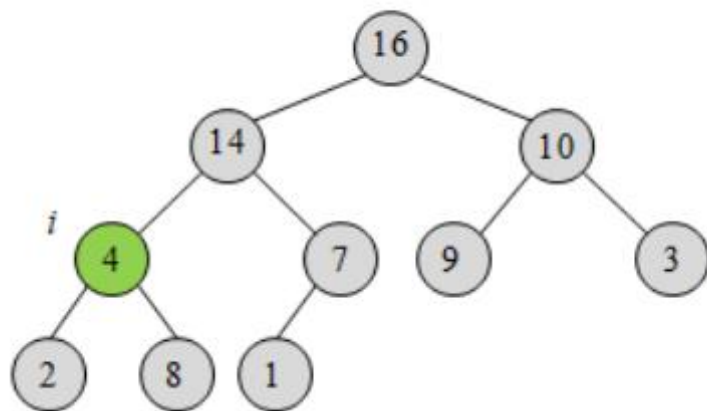
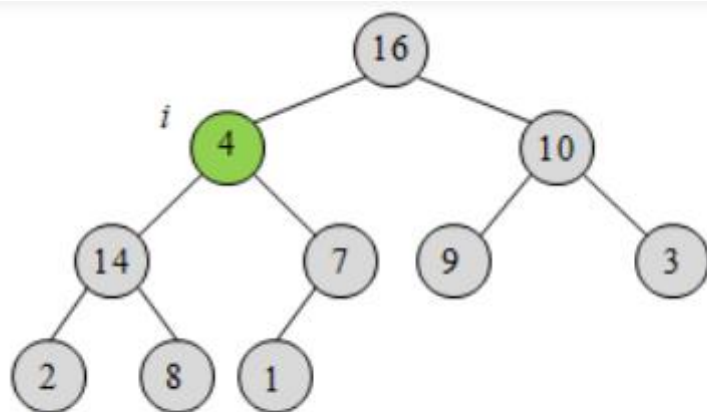
优先队列需求



优先队列的操作:

- 1) $\text{Insert}(S, x)$: 插入 x 到集合 S 中;
- 2) $\text{Maximum}(S)$: 返回 S 中具有最大 (或者最小) 关键字的元素;
- 3) $\text{Extract_Max}(S)$: 去掉并返回集合 S 中具有最大 (或者最小) 关键字的元素;
- 4) **$\text{Increase_Key}(S, x, \text{key})$: 将元素 x 的关键字增加到 key 。**

维护堆的性质



<https://blog.csdn.net/yangtzhou>



从堆中改变元素大小复杂度

最坏情况就是改变根然后自顶向下下沉到堆的叶子（或者改变叶子然后自底向上更新到根），于是时间代价为堆的深度 k
因此，从改变元素值的时间代价为 $O(\log n)$

建堆



建堆方法:

- ✓ 自底向上调整建堆
- ✓ 自顶向下插入建堆



建堆方法1：自底向上（迭代实现）

代码 6.3-1：构建最大堆

//参数A：一个用来构建最大堆的数组

```
1 BUILD-MAX-HEAP(A,i)
2   A.heap_size = A.length
3   r = RIGHT (i)
4   for i =  $\lfloor A.length/2 \rfloor$  downto 1
5     MAX-HEAPIFY(A,i)
```

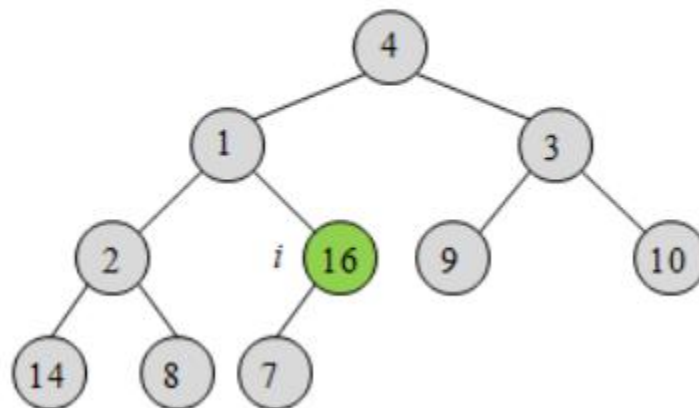
建堆



建堆方法1：自底向上（迭代实现）

原始数组 A

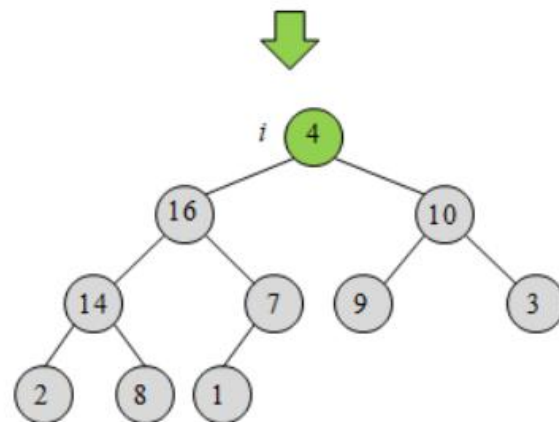
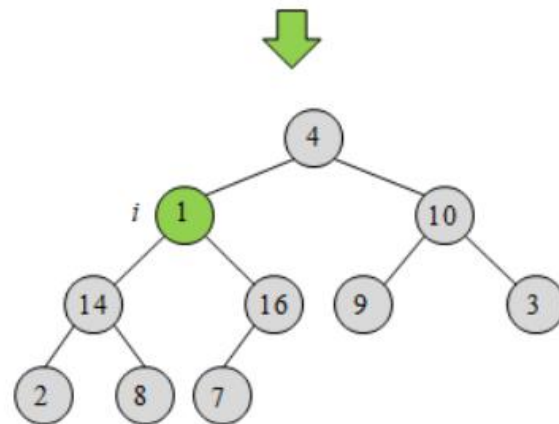
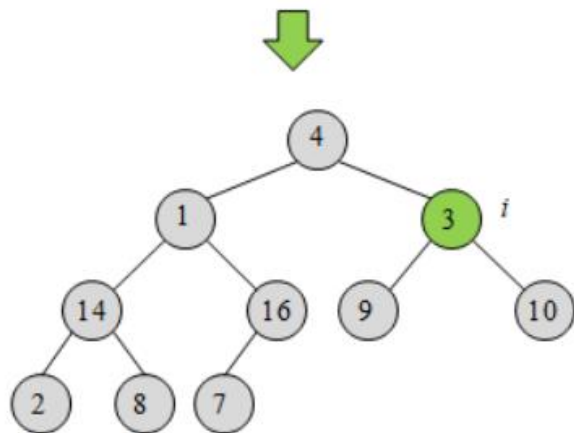
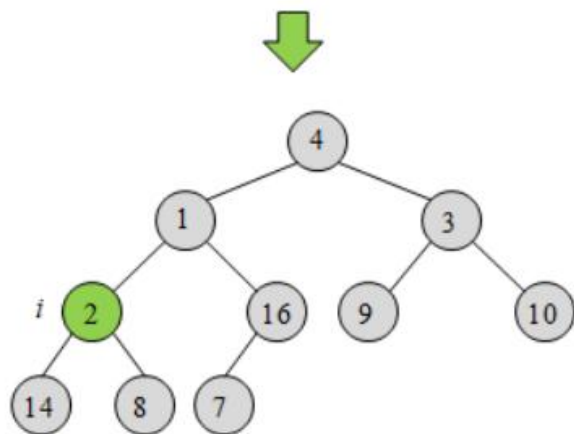
4	1	3	2	16	9	10	14	8	7
---	---	---	---	----	---	----	----	---	---



建堆



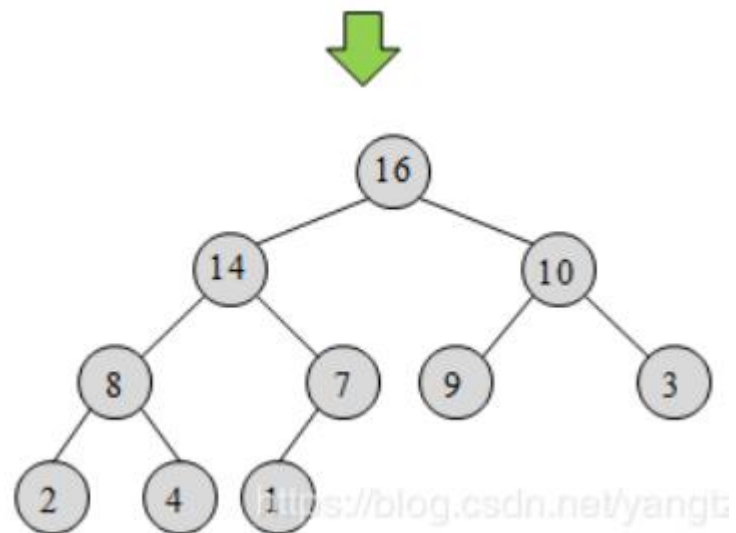
建堆方法1：自底向上（迭代实现）



建堆



建堆方法1：自底向上（迭代实现）





建堆方法2：插入建堆

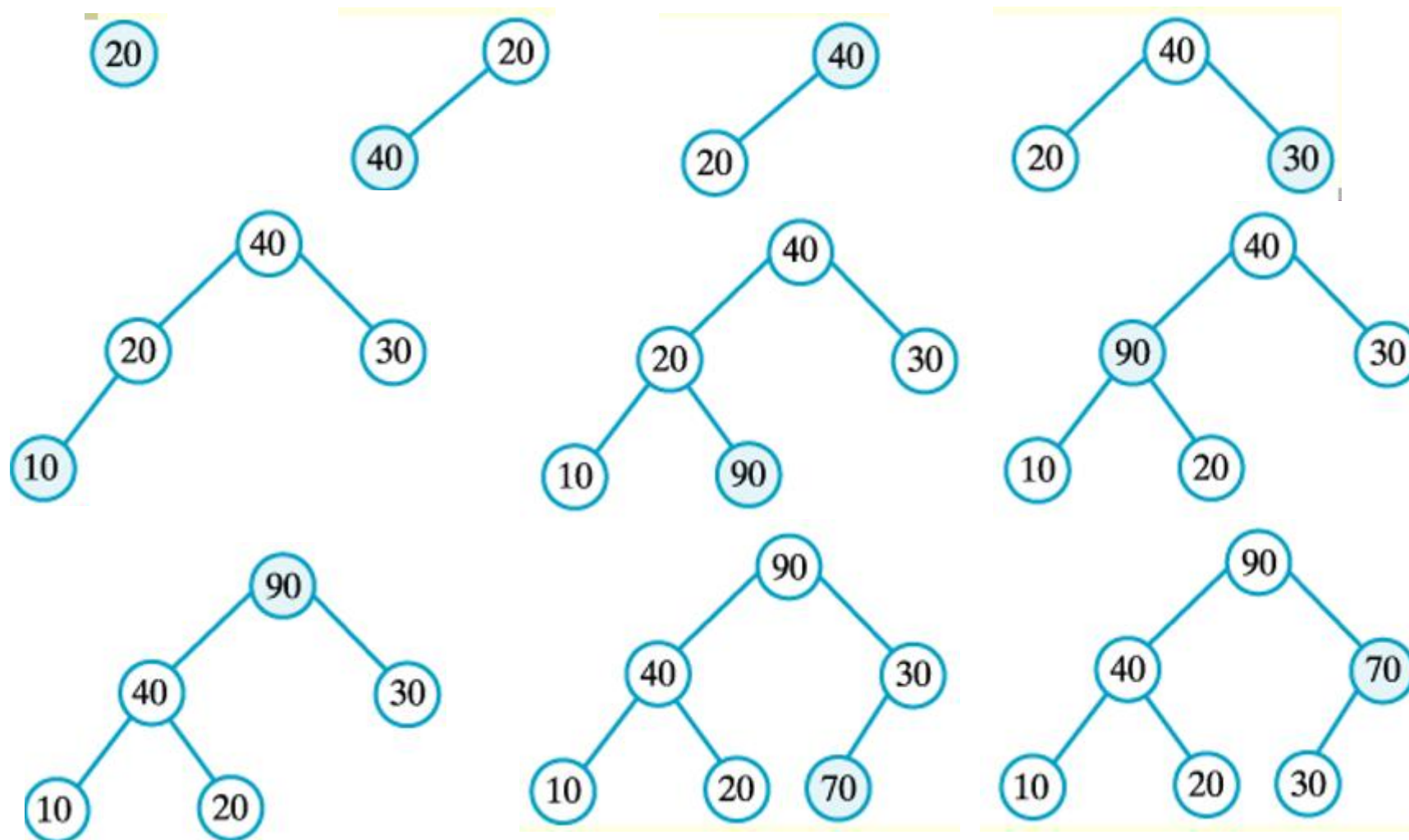
把一个数组看成两部分，左边是堆，右边是还没加入堆的元素，步骤如下：

- 1、数组里的第一个元素自然地是一个堆
- 2、然后从第二个元素开始，一个个地加入左边的堆，当然，每加入一个元素就破坏了左边元素堆的性质，得重新把它调整为堆。

建堆



建堆方法2:
插入建堆





建堆的复杂度

最坏情况就是，于是每个元素都是自底向上从叶子到根（或者自顶向下从根到叶子），每个元素调整代价为 $O(\log n)$

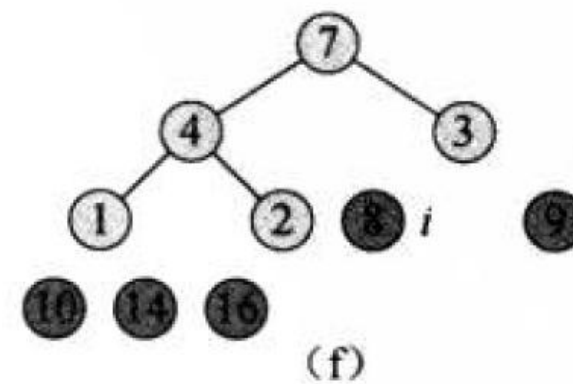
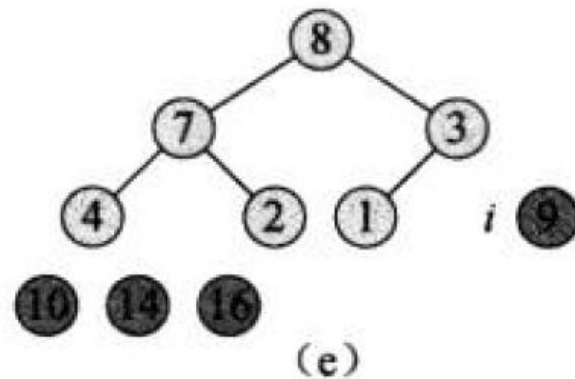
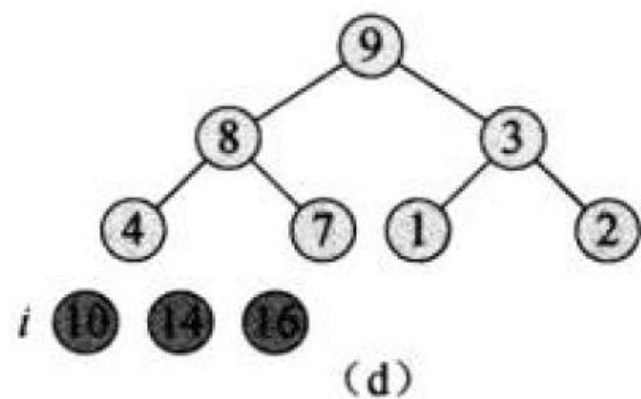
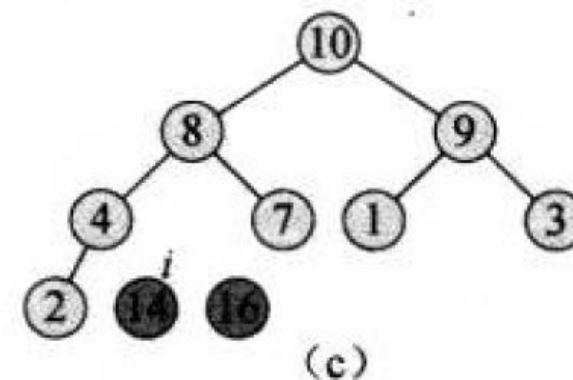
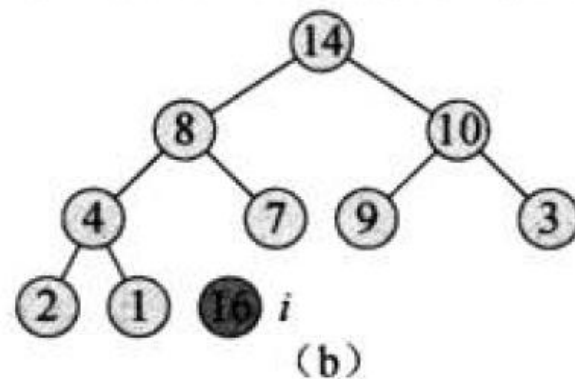
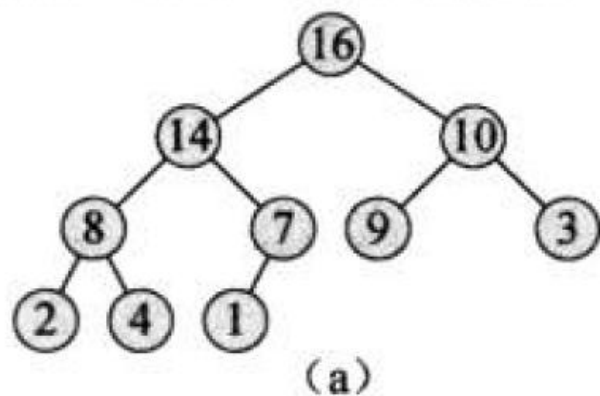
因此，对于 n 个元素，建堆时间代价为 $O(n \log n)$

堆排序算法



若在输出堆顶的最小值（最大值）后，使得剩余 $n-1$ 个元素的序列重又建成一个堆，则得到 n 个元素的次小值（次大值）.....如此反复，便能得到一个有序序列，这个过程称之为**堆排序算法**。

堆排序算法



堆排序算法



堆排序的时间复杂度

堆排序=堆调整 + 堆排序

堆排序时间复杂度=堆调整时间复杂度 + 堆排序时间复杂度

堆调整的时间复杂度为 $O(\log n)$ ，需要迭代 n 轮

总的时间复杂度为 $O(n \lg n)$

目录

第一节

堆

第二节

哈希表

快速查找问题



手机上都有基本的通讯功能，有通讯功能必然存在通讯录

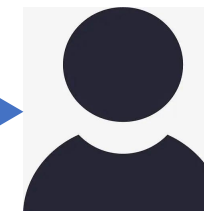
通讯录功能： 输入姓名能映射出电话号码，输入电话号码能映射出姓名



1234 5678



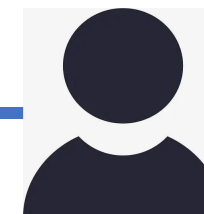
张三



8765 4321



李四



哈希表的作用



哈希表是一种基于数组实现的数据结构。它通过把关键码值映射到数组中一个位置来访问记录，存放记录的数组叫做哈希表。
这个映射函数叫做**哈希函数**。

哈希表的应用场景：

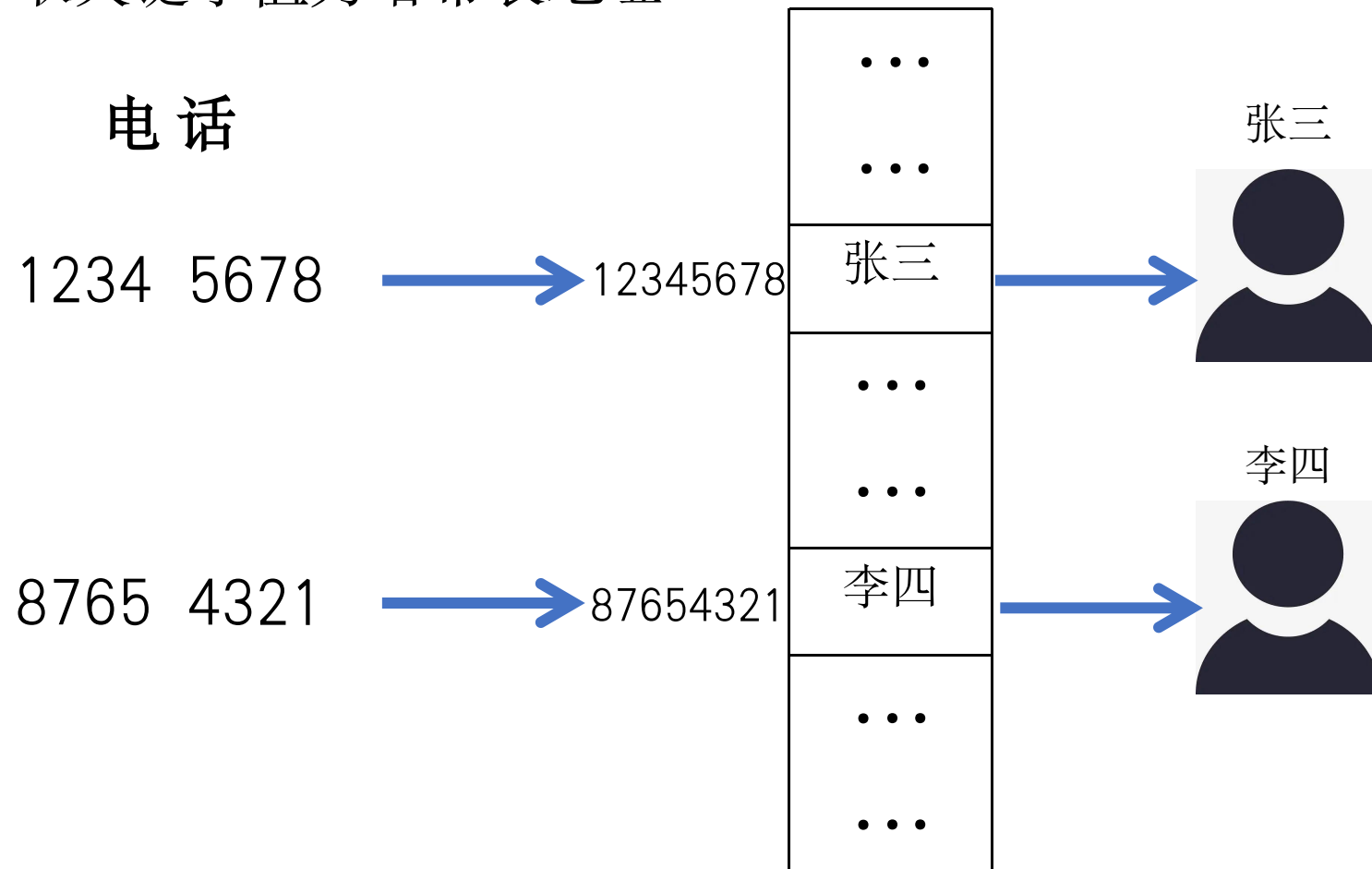
- 网页快速查找信息
- DNA匹配：字符串之间的匹配

课程目标： 哈希表定义以及常用哈希函数的定义和计算，如除法哈希

直接寻址



直接寻址：取关键字值为哈希表地址



直接寻址优缺点



优点：查找速度快

缺点：对于全域较大，但是元素却十分稀疏的情况，使用这种存储方式将浪费大量的存储空间。

哈希函数



为了克服直接寻址技术的缺点，而又保持其快速字典操作的优势，我们可以利用哈希函数 $h: U \rightarrow \{0, 1, 2, \dots, m-1\}$ 来计算关键字 k 所在的位置

哈希函数 $h(k)$ 的作用是将范围较大的关键字映射到一个范围较小的集合中。形式化来说，设关键字集 K 中有 n 个关键字，哈希表长为 m ，哈希函数 h 满足以下要求：

1. 对任意 $k \in K$ ，有 $0 \leq h(k) \leq m-1$ ；
2. 对任意 $k \in K$ ， $h(k)$ 取 $[0, m-1]$ 中任一值的概率相等，即每个值取到的概率都是 $1/m$ 。

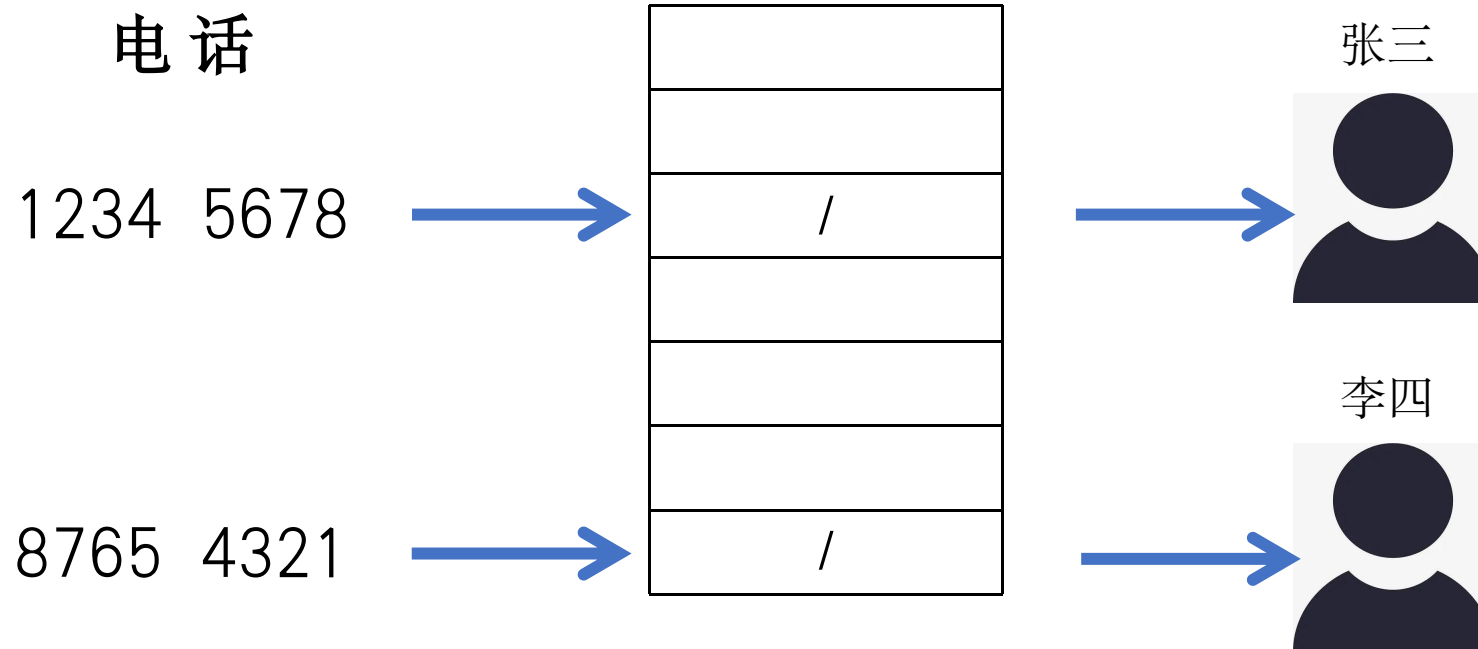
除法哈希函数



假设 m 是哈希表大小，通过取关键字值 k 除以 m 的余数，将关键字值 k 映射到哈希表中的一个位置

$$h(k) = k \bmod m$$

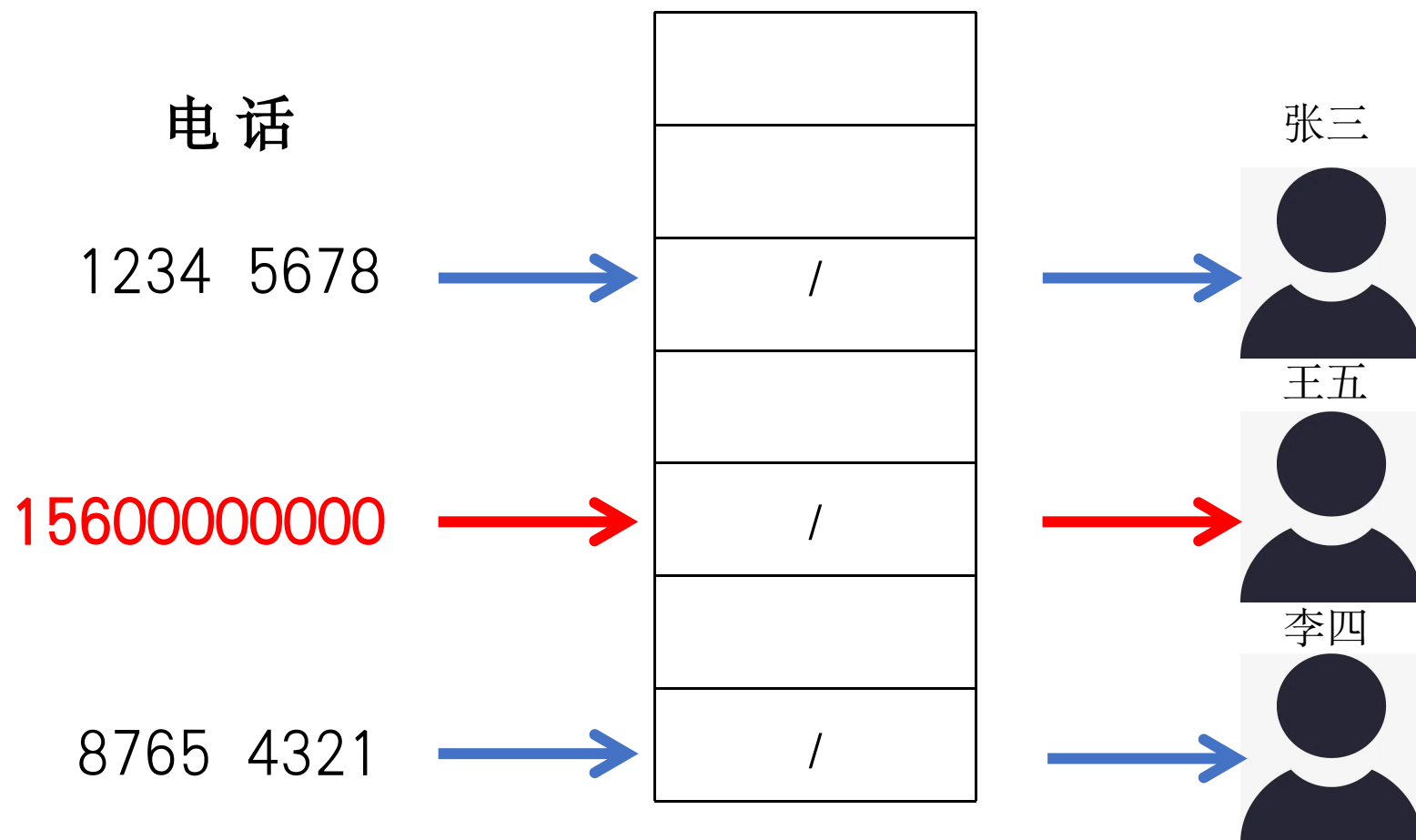
假设 $m=7$



除法哈希函数



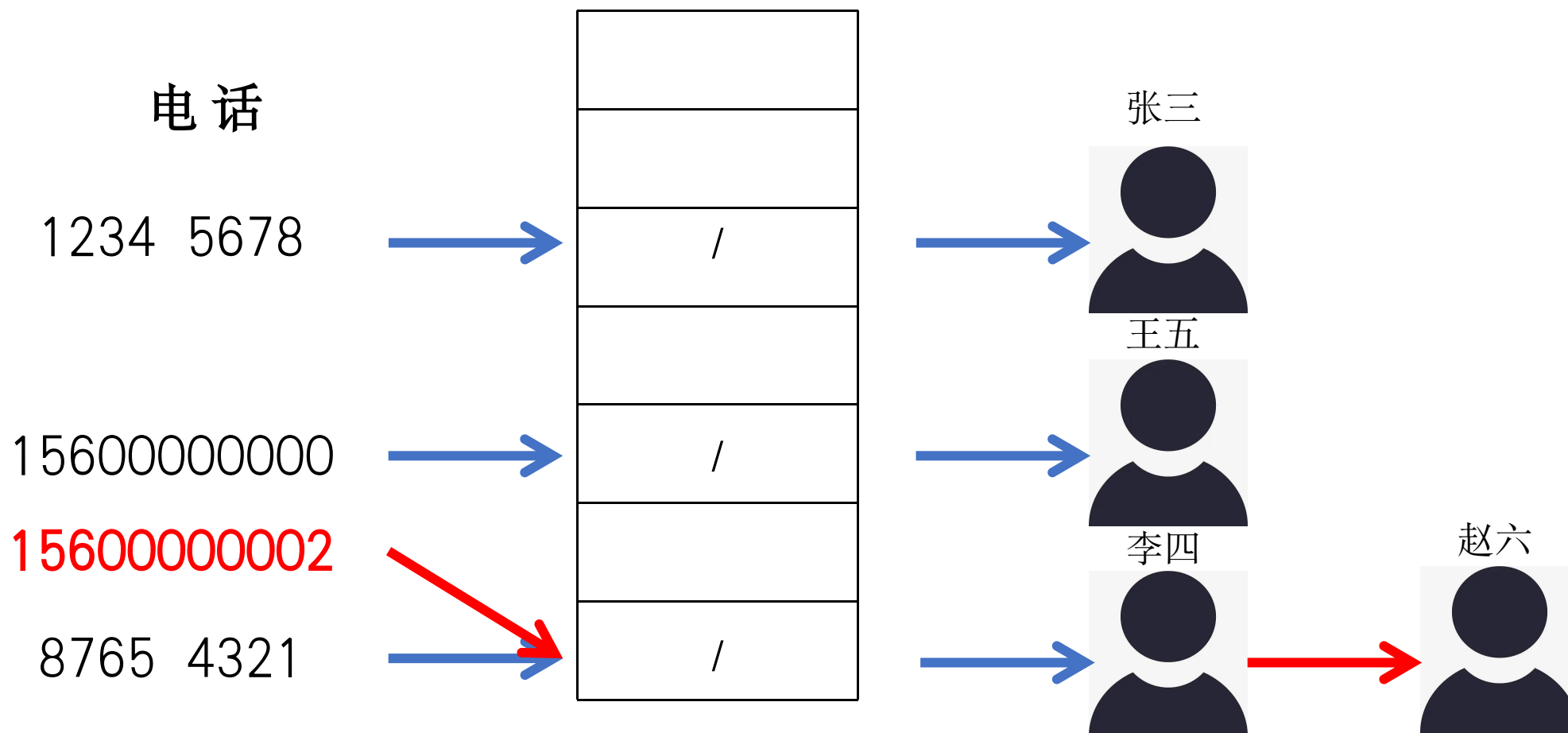
计算电话为156000000000的王五在上述哈希表中位置



哈希函数的核心挑战——冲突



计算电话为15600000002的赵六在上述哈希表中位置



除法哈希函数的不足



- ❑ 针对具有规律性的键值集合，有规律性问题：
 - 假设键值为 $x, 2x, 3x, 4x, \dots$ （例如：若 x 与 m 存在公约数 d ）
 - 若 x 与所选模数 m 有公共因子 d ，则哈希表仅能利用其 $1/d$ 的空间：
 - ✓ x 的倍数序列会循环映射，导致 $d-1$ 个槽位始终为空；
 - ✓ 举例：若 m 是 2 的幂，而所有键均为偶数 $\rightarrow$ 仅使用一半空间。
- ❑ 解决方案：令 m 为质数但寻找大质数本身计算成本较高；
 - 此外，除法运算效率低下（相比乘法或位移操作慢得多）。

乘法哈希



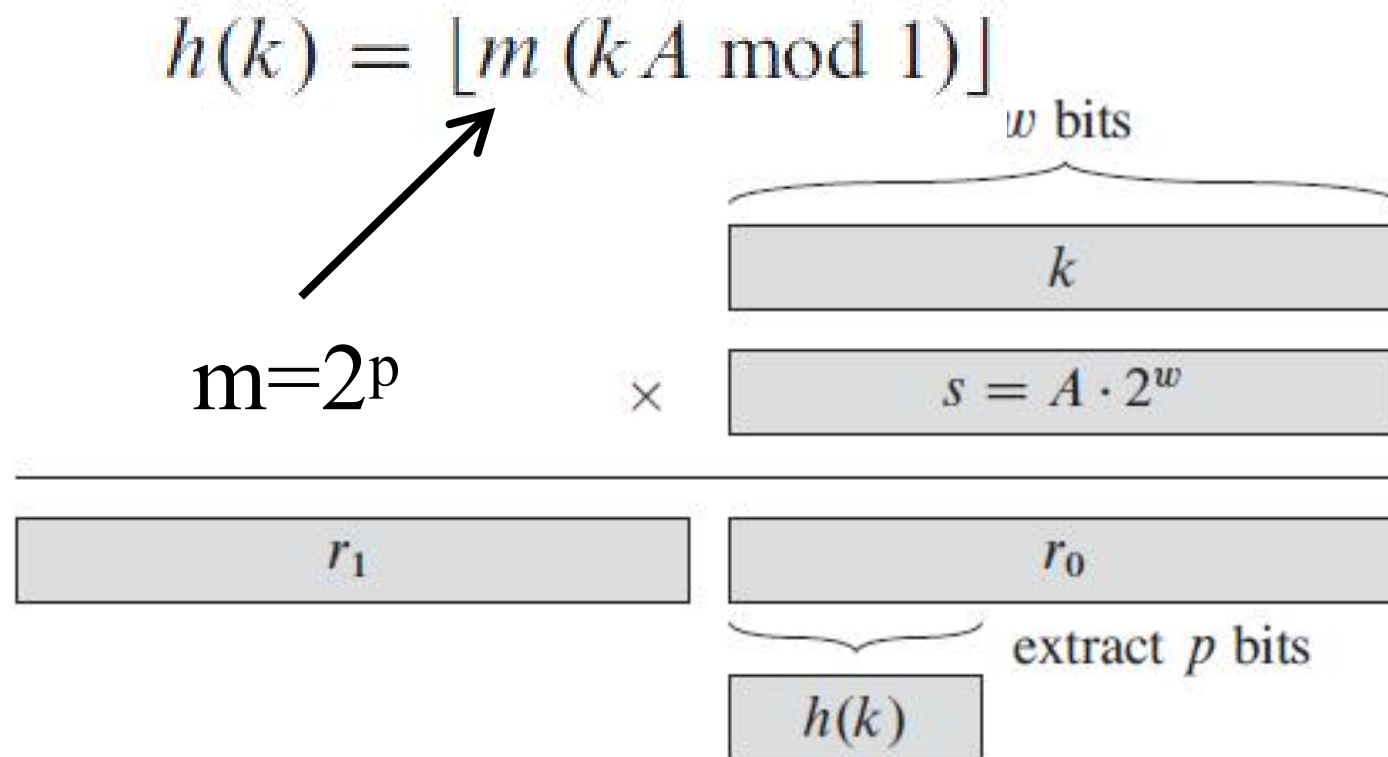
原理：将键值 k 乘以一个常数 A ($0 < A < 1$)，提取结果的小数部分，再乘以表长 m 并取整。

优点：

1. 速度快：若 $m=2^p$ ，可用位移操作替代除法。
2. 鲁棒性强： A 选得好，对任意 m 效果均佳。

缺点：

1. 涉及浮点运算（在某些嵌入式环境可能较慢）。
2. 相比除留余数法，对质数 m 的依赖性低但常数 A 需精心挑选。



$$A \approx (\sqrt{5} - 1) / 2 = 0.618033988$$

哈希函数：哈希值



Key不是数字，是字符串呢？

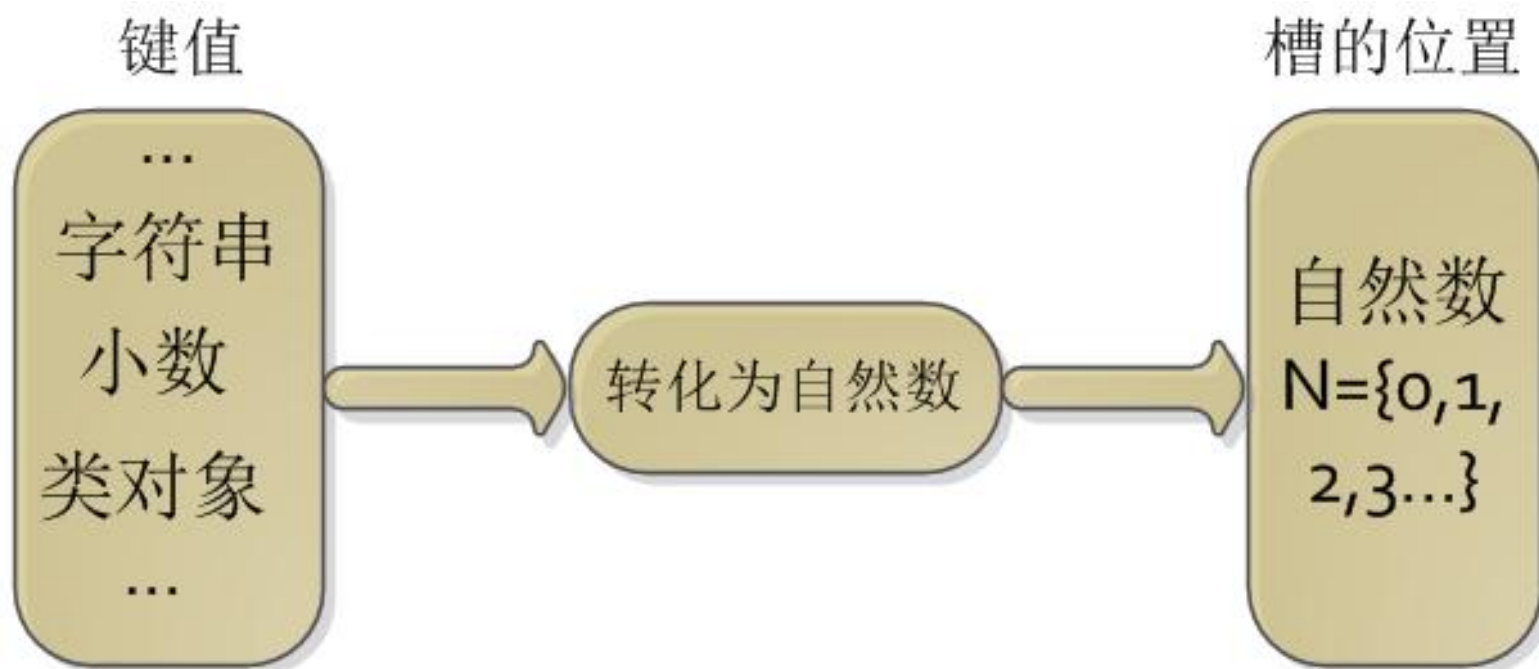
Key="abcdef"



Hash值



哈希函数:哈希值



Java: hashCode

C++: ASCII码相加

Python:

Key="abcdef"

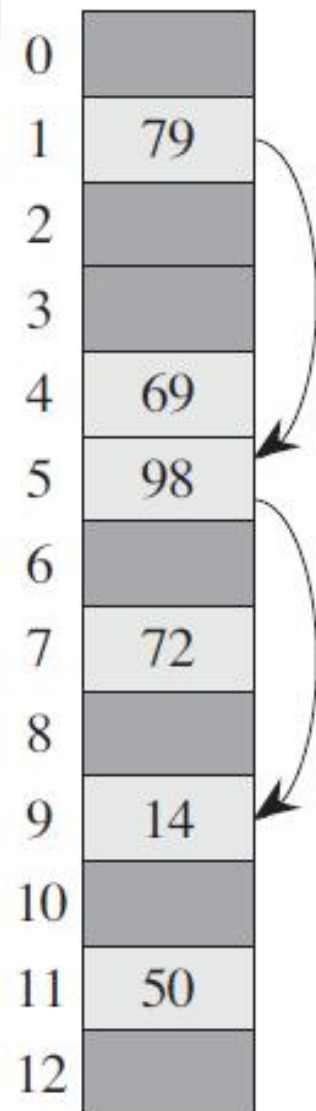
A=hash(Key)

开放寻址法



开放寻址法（open addressing）中，所有元素都存放在槽中，在链表法哈希表中，每个槽中保存的是相应链表的指针，为了维护一个链表，链表的每个结点必须有一个额外的域来保存它的前驱和后继结点。

开放寻址法



开放寻址法



三种常用技术来计算开放寻址法中的探查序列

1. 线性探查
2. 二次探查
3. 双重哈希

开放寻址法-线性探查



$$h(k, i) = (h'(k) + i) \bmod m$$

关键词 (key)	47	7	29	11	9	84	54	20	30
散列地址 $h(\text{key})$	3	7	7	0	9	7	10	9	8
冲突次数	0	0	1	0	0	3	1	3	6

开放寻址法-线性探查



关键词 (key)	47	7	29	11	9	84	54	20	30
散列地址 $h(\text{key})$	3	7	7	0	9	7	10	9	8
冲突次数	0	0	1	0	0	3	1	3	6

地址 操作	0	1	2	3	4	5	6	7	8	9	10	11	12	说明
插入47				47										无冲突
插入7				47				7						无冲突
插入29				47				7	29					$d_1 = 1$
插入11	11			47				7	29					无冲突
插入9	11			47				7	29	9				无冲突
插入84	11			47				7	29	9	84			$d_3 = 3$
插入54	11			47				7	29	9	84	54		$d_1 = 1$
插入20	11			47				7	29	9	84	54	20	$d_3 = 3$
插入30	11	30		47				7	29	9	84	54	20	$d_6 = 6$

注意“聚集”现象

http://blog.csdn.net/qq_30091945

开放寻址法-二次探查



$$h(k, i) = (h'(k) + c_1 \times i + c_2 \times i^2) \bmod m$$

开放寻址法-双重哈希



$$h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$$

Cuckoo Hash 布谷鸟哈希



定义：一种解决 hash 冲突的方法，其目的是使用简单的 hash 函数来提高 hash table 的利用率，同时保证 $O(1)$ 的查询时间。基本思想是使用2个 hash 函数来处理碰撞，从而每个 key 都对应到2个位置。

Cuckoo Hash 布谷鸟哈希



操作:

- 1) 对 key 值 hash, 生成两个 hash key 值, hashk_1 和 hashk_2 , 如果对应的两个位置上有一个为空, 那么直接把 key 插入即可。
- 2) 否则, 任选一个位置, 把 key 值插入, 把已经在那个位置的 key 值踢出来。
- 3) 被踢出来的 key 值, 需要重新插入, 直到没有 key 被踢出为止。

Cuckoo Hash 布谷鸟哈希



衍生背景:

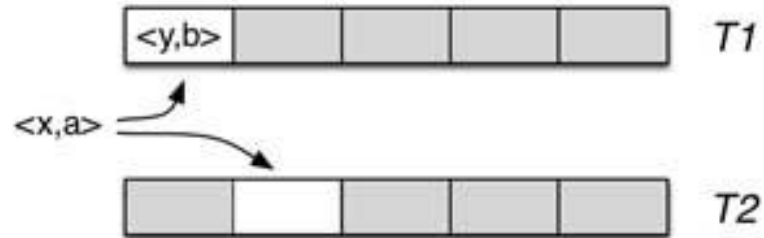
Cuckoo中文名叫布谷鸟，这种鸟有一种即狡猾又贪婪的习性，它不肯自己筑巢，而是把蛋下到别的鸟巢里，而且它的幼鸟又会比别的鸟早出生，布谷幼鸟天生有一种残忍的动作，幼鸟会拼命把未出生的其它鸟蛋挤出窝巢，今后以便独享“养父 母”的食物。借助生物学上这一典故，cuckoo hashing处理碰撞的方法，就是把原来占用位置的这个元素踢走，不过被踢出去的元素还要比鸟蛋幸运，因为它还有一个备用位置可以安置，如果备用位置上 还有人，再把它踢走，如此往复。直到被踢的次数达到一个上限，才确认哈希表已满，并执行rehash操作。

Cuckoo Hash 布谷鸟哈希



Insertion when one of the two buckets is empty

Step 1: Both buckets for $\langle x, a \rangle$ are tested, the one in T2 is empty.

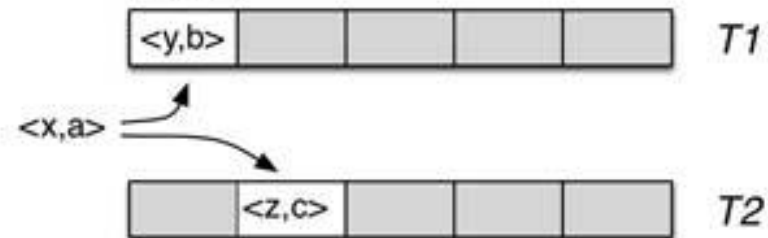


Step 2: $\langle x, a \rangle$ is stored in the empty bucket in T2.

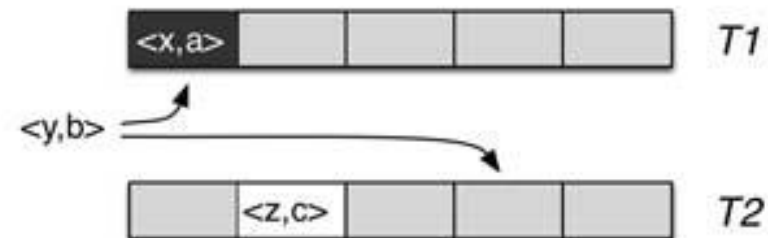


Insertion when the two buckets already contain entries

Step 1: Here $\langle y, b \rangle$ will be withdrawn from T1 so that $\langle x, a \rangle$ can be stored.



Step 2: After $\langle x, a \rangle$ has been stored in T1, $\langle y, b \rangle$ needs to be moved to T2. The bucket in T2 may already contain an entry, if so this entry will need to be moved.



Cuckoo Hash 布谷鸟哈希



其他:

- 1) Cuckoo hash 有两种变形。一种通过增加哈希函数进一步提高空间利用率；另一种是增加哈希表，每个哈希函数对应一个哈希表，每次选择多个表中空余位置进行放置。三个哈希表可以达到80%的空间利用率。
- 2) Cuckoo hash 的过程可能因为反复踢出无限循环下去，这时候就需要进行一次循环踢出的限制，超过限制则认为需要添加新的哈希函数。

哈希的扩展应用



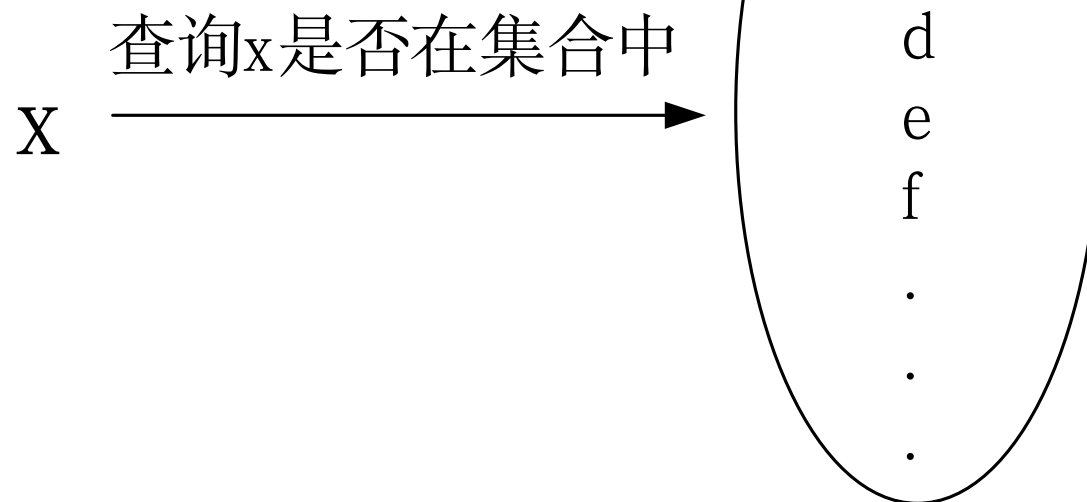
- 布鲁姆过滤器 (BloomFilter)
- 最小哈希
- 区块链

布鲁姆过滤器 (bloomfilter)



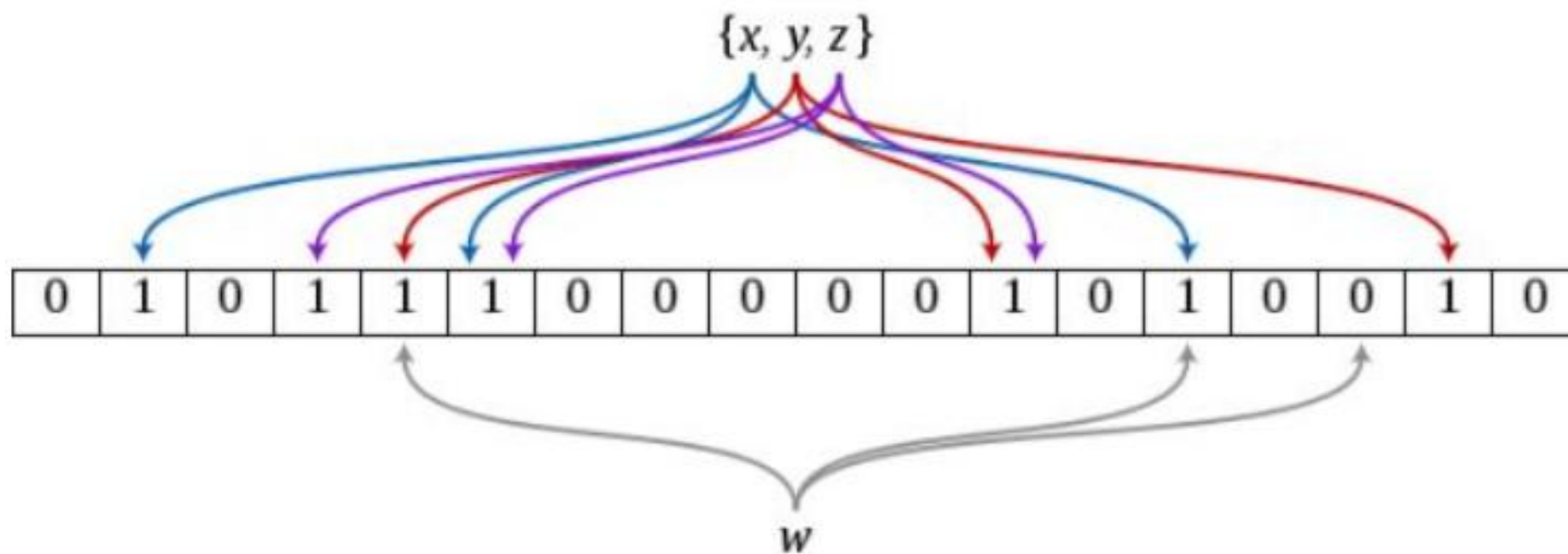
问题1

数据集



10亿数据

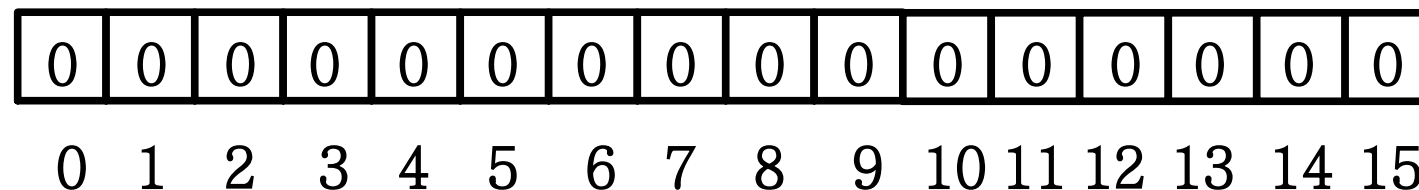
布鲁姆过滤器 (bloomfilter)



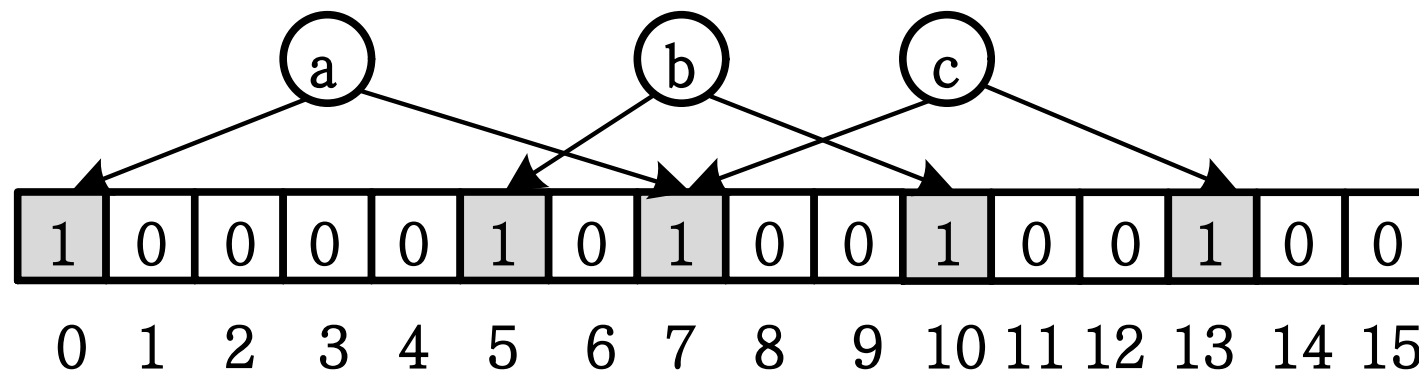
布鲁姆过滤器 (bloomfilter)



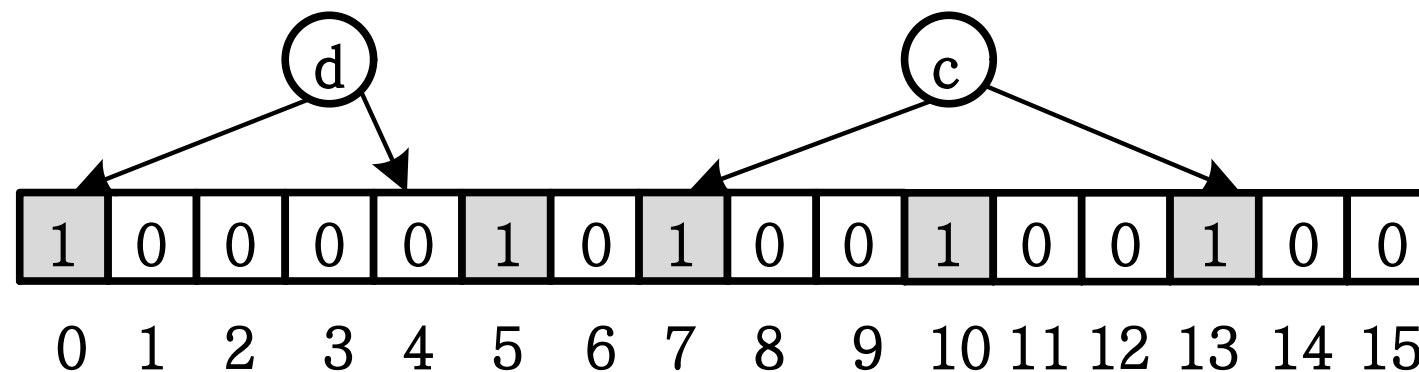
初始化



插入a, b, c



查询d, c





布鲁姆过滤器算法关键指标

- 空间复杂度
- 时间复杂度
- 误判率 (假阳性)

将不属于集合的元素误判断成属于集合中的这种假阳性误判称为一次误判。发生误判的概率称为误判率。

布鲁姆过滤器 (bloomfilter)



误判率分析 (假阳性)

理论分析, 假设 k 个哈希函数、 m 位bitset, n 个元素之后

$$f_{\text{BF}}(m, k, n) = (1 - e^{-k \cdot n / m})^k$$

布鲁姆过滤器 (bloomfilter)



误判率分析 (假阳性)

最优分析

$$k = \lceil \ln 2 (m / n) \rceil$$

$$k_{\min} = (\ln 2) \left(\frac{m}{n} \right)$$

布鲁姆过滤器 (bloomfilter)



误判率分析 (假阳性)

(1) 理论分析, 假设 k 个哈希函数、 m 位bitset, n 个元素之后

$$f_{\text{BF}}(m, k, n) = (1 - e^{-k \cdot n / m})^k$$

最小哈希



MinHash是一种局部敏感哈希技术
可以用来快速估算两个集合的相似度

Jaccard相似度



Jaccard相似度是用来计算集合相似性，也就是距离的一种度量标准

假如有集合A、B，那么 $J(A,B) = |A \cap B| / |A \cup B|$

最小哈希



$h(x)$: 把 x 映射成一个整数的哈希函数

$hmin(S)$: 集合 S 中的元素经过 $h(x)$ 哈希后, 具有最小哈希值的元素

那么对集合 A 、 B , $hmin(A) = hmin(B)$ 成立的条件是 $A \cup B$ 中具有最小哈希值的元素也在 $A \cap B$ 中

最小哈希性质



假设 $h(x)$ 是一个良好的哈希函数，它具有很好的均匀性，能够把不同元素映射成不同的整数。

所以有， $\Pr[hmin(A) = hmin(B)] = J(A,B)$ ，即集合A和B的相似度为集合A、B经过hash后最小哈希值相等的概率。

区块链典型应用——比特币



货币的特点:

1. 去中心化, 交易不经手中央银行
2. 匿名性, 交易双方之间可以互相匿名

比特币就是为了在网络上实现货币, 为此增加一个要求——不可篡改

区块链基础——加密的哈希函数



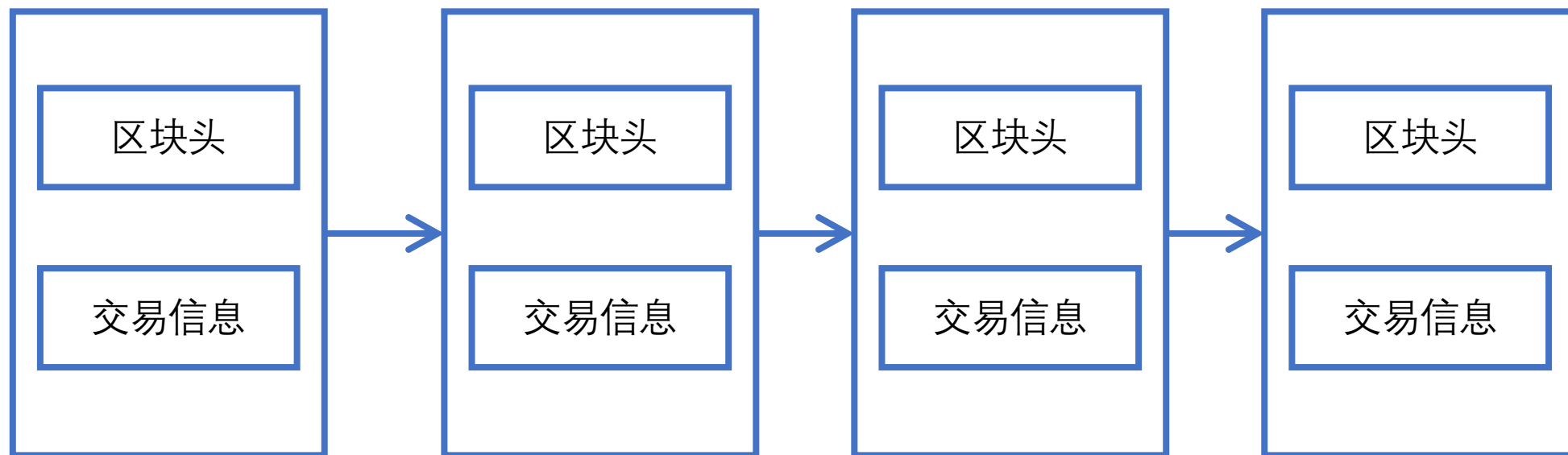
哈希函数广泛应用于信息安全，特别是保护密码。当您在网站上注册帐户时，您的密码将通过哈希函数进行转换，从而生成哈希摘要，然后由服务存储。登录后，您输入的密码将经过相同的哈希函数，并将生成的哈希值与存储的哈希值进行比较，以验证您的身份。

加密哈希函数就是实现了从哈希中破译原始密码的哈希函数。这样增强了安全性，因为即使黑客利用哈希摘要访问数据库也无法获得密码

区块链数据结构



逻辑上它是一个链式(chain)结构，每个结点上就是一个区块信息(block)，区块里面则存储了交易的信息。



区块链数据结构



区块链数据结构主要包括为区块头和区块体两部分

区块的哈希值是利用加密哈希函数对区块头中信息进行计算而得到的。

这个哈希值是区块的标识符，可以通过这个哈希值找到对应的区块，加密哈希函数保证了匿名。

当前区块中包含来前一个区块的哈希值，如此形成了一个链表。通过这个链表可以溯源整个交易来验证交易，保证了不可篡改。

比特币投票机制



比特币网络投票原理是在比特币网络中实现的一种特殊形式的网络投票。比特币网络中的投票是为了决定关键事务的变更，例如是否接受某个交易、是否添加某个区块等。

投票机制保证了去中心化



湖南大学
HUNAN UNIVERSITY

谢谢观赏

—— 实事求是 敢为人先 ——